

Технологический стек

- **Backend:**

- Язык: Java (Java 21)
- Фреймворк: Spring Boot 3.x
- База данных: PostgreSQL (основная база)
- Кэширование: Redis (для ускорения доступа к часто используемым данным)
- Контроль версий: Git (репозиторий на GitHub или аналогичный сервис)

- **Frontend:**

- Язык: JavaScript
- Библиотека: React
- Дополнительные библиотеки: Axios (для работы с HTTP-запросами), React Router (для маршрутизации)

Используемые паттерны проектирования

Для повышения гибкости и расширяемости проекта применяются следующие паттерны:

- **Factory:** Для создания различных типов счетов (например, DebitAccount, SavingsAccount, DebtAccount) через единый интерфейс.
- **Builder:** Для пошагового создания объектов с множеством параметров (например, при создании сложных DTO).
- **Repository:** Для абстрагирования доступа к данным (интерфейсы AccountRepository, CategoryRepository, TransactionRepository), что позволяет легко переключаться между реализациями хранения данных.
- **DTO (Data Transfer Object):** Для передачи данных между слоями, не нарушая инкапсуляцию доменной модели.
- **Singleton:** Для настройки и управления подключением к Redis, чтобы обеспечить единообразный доступ к кэшу.
- **Clean Architecture / Onion Architecture:** В качестве общего принципа организации кода для обеспечения слабой связанности и высокой тестируемости.

Архитектурный паттерн

Выбранная архитектура: Onion Architecture (Чистая архитектура)

Обоснование:

Onion Architecture позволяет отделить бизнес-логику от инфраструктурных зависимостей, что облегчает тестирование, поддержку и расширение приложения. Основные слои архитектуры:

- **Domain Layer:** Содержит основные сущности и бизнес-правила. Здесь располагаются POJO (например, Account, Category, Transaction), которые не содержат бизнес-логики, а лишь представляют данные.
- **Application Layer:** Отвечает за реализацию бизнес-логики и use-case'ов (сервисы, например, TransactionService, AccountService, CategoryService).
- **Infrastructure Layer:** Реализует доступ к внешним ресурсам (работа с базой данных через репозитории, интеграция с Redis, API клиентов).
- **API Layer (Web):** Представляет REST-контроллеры и DTO, обеспечивающие взаимодействие с внешними клиентами (например, AccountController, TransactionController, CategoryController).